UNIVERSIDAD PRIVADA DEL NORTE
Lenguajes de Programación — UAIN1288P

MedicRural-PE

INFORME TÉCNICO FINAL

INTEGRANTES

Jeremy Leonel Farfan Villa
German Alexander Flores Ponce
Eduard David Gozme Flores
Sebastian Joaquin Huaman Quispe
Gonzalo Fabian Portillo Nieto

DOCENTE

Williams Alfonso Chullo Villagaray

Periodo académico 2026-II

1. Introducción y contexto del problema

1.1 ¿Qué problema queremos resolver?

Pensemos en una persona que debe revisar las existencias de un pequeño establecimiento de salud. Tiene una lista de medicamentos e insumos, pero necesita responder tres preguntas sencillas: ¿qué productos hay?, ¿cuántas unidades quedan? y ¿cuáles necesitan atención primero? Si el registro se lleva de forma poco ordenada, responder puede tomar tiempo y es más fácil pasar por alto una cantidad baja. Este es el problema concreto que elegimos estudiar.

MedicRural-PE nació como una respuesta académica a esa situación. Es una aplicación de escritorio que permite registrar productos, consultar su cantidad disponible y reconocer aquellos que llegan al mínimo establecido. No pretende resolver toda la administración de un centro de salud: elegimos el inventario porque es una parte delimitada del desafío del curso y permite demostrar, con un programa funcional, varios conceptos de programación (Universidad Privada del Norte [UPN], 2026a).

Es importante aclarar el contexto: el establecimiento descrito es un escenario de trabajo simulado, no una institución visitada o una entidad que haya utilizado el software. Tampoco realizamos entrevistas ni probamos el sistema con pacientes o trabajadores. Por eso, este informe muestra lo que construimos y lo que pudimos comprobar, sin atribuirle resultados obtenidos en un centro de salud real.

1.2 Qué hace el sistema y hasta dónde llega

El propósito del prototipo es organizar medicamentos e insumos mediante datos básicos: código, nombre, tipo, cantidad actual, cantidad mínima y una característica adicional (presentación del medicamento o unidad de medida del insumo). Por ejemplo, si registramos 10 unidades de paracetamol y fijamos un mínimo de 20, el sistema lo señala como crítico. Además, puede mostrar solo los recursos críticos y ordenarlos por su faltante para facilitar la revisión.

El programa también permite actualizar existencias y obtener un resumen con el número de recursos, cuántos están en estado crítico y la suma de unidades registradas. Esa suma sirve como conteo general; no se debe interpretar como una unidad física única, porque puede reunir tabletas, cajas u otras presentaciones. No gestiona pacientes, citas, diagnósticos, compras ni recetas. Tampoco guarda la información al cerrar: actualmente trabaja solo en memoria.

La tabla 1 reúne esas siete funciones y muestra qué parte del programa se ocupa de cada una. Así podemos pasar de una necesidad concreta a una evidencia dentro del código.

Tabla 1. Funciones que debe cumplir MedicRural-PE

Código	Función explicada en palabras sencillas	Dónde se realiza
RF01	Registrar un medicamento o insumo con sus datos básicos.	RecursoFactory e Inventario
RF02	Ver todos los productos agregados al inventario.	Inventario y FrmMedicRural
RF03	Cambiar la cantidad disponible de un producto usando su código.	Inventario.actualizarStock()
RF04	Identificar si la cantidad llegó al mínimo o está por debajo.	RecursoMedico.esCritico()
RF05	Mostrar únicamente los productos críticos.	AnalizadorInventario
RF06	Ordenar los críticos según cuánto les falta.	priorizarRecursosCriticos()
RF07	Mostrar un resumen de los registros y cantidades.	mostrarResumen() y analizador

1.3 Reglas de trabajo y criterios de éxito

Para que el programa sea sencillo de ejecutar y de explicar, adoptamos cinco restricciones. RD01: utilizar Java OpenJDK, NetBeans y Swing para crear una aplicación de escritorio. RD02: guardar los registros solo en una lista en memoria, aceptando que se perderán al cerrar. RD03: trabajar con datos ficticios de productos y no solicitar información de pacientes o responsables. RD04: separar las clases de los recursos, la gestión del inventario, el análisis y la interfaz. RD05: mantener el código en Git/GitHub para registrar sus cambios. Estas decisiones acotan el trabajo; no son características de un sistema hospitalario listo para operar.

También consideramos requisitos de calidad: RNF01, avisar cuando un dato es inválido; RNF02, permitir el uso local sin conexión para operar el inventario; RNF03, distribuir responsabilidades para facilitar cambios; y RNF04, minimizar datos personales. La decisión de no solicitar identificadores es proporcional al alcance académico y está relacionada con la protección de datos que exige el desafío y con la Ley N.° 29733 (Congreso de la República del Perú, 2011).

Definimos tres criterios medibles para saber si las reglas principales funcionan: CE01, un producto con 10 unidades y un mínimo de 20 debe ser crítico; CE02, el sistema debe rechazar un segundo producto que repita el código TEST001; CE03, si un producto necesita 25 unidades y otro necesita 12, el de 25 debe aparecer primero.

2. Análisis comparativo de paradigmas y selección justificada

2.1 Tres maneras de abordar un mismo problema

Un paradigma de programación es una forma de organizar las instrucciones de un programa. No tenemos que elegir uno para todo: podemos combinar enfoques cuando cada uno resuelve una necesidad distinta. En MedicRural-PE usamos orientación a objetos para representar los productos, programación funcional para consultar listas y programación orientada a eventos para responder a los botones de la ventana. La tabla 2 compara sus ventajas y dificultades, tal como pide el desafío (UPN, 2026a).

Tabla 2. Comparación de los tres paradigmas considerados

Paradigma	Qué nos aporta	Qué debemos cuidar	Ejemplo en el proyecto
Orientado a objetos	Agrupa los datos de cada producto con las reglas que lo validan.	Más clases no siempre significan una mejor solución.	RecursoMedico, Medicamento e InsumoMedico.
Funcional	Permite filtrar, ordenar y sumar los registros con operaciones claras.	Las expresiones lambda requieren aprender a leerlas.	filter, map, reduce y sorted en AnalizadorInventario.
Orientado a eventos	Hace que el programa actúe cuando alguien presiona un botón.	La interfaz puede volverse confusa si contiene toda la lógica.	Botones y eventos ActionPerformed de FrmMedicRural.

Nota. Los tres enfoques tienen usos reales en el código; no se agregaron para completar una cantidad.

2.2 Por qué elegimos combinarlos

La primera razón fue representar correctamente los dos tipos de productos (RF01 y RF04). Ambos tienen código, nombre y cantidades; por eso comparten una clase base. La segunda fue trabajar con una lista de recursos sin escribir una búsqueda diferente para cada consulta (RF05–RF07). Allí resultan útiles las funciones de filtrado, ordenación y suma. La tercera fue permitir que cualquier usuario interactúe con un formulario y botones, sin tener que escribir comandos (RF01–RF03).

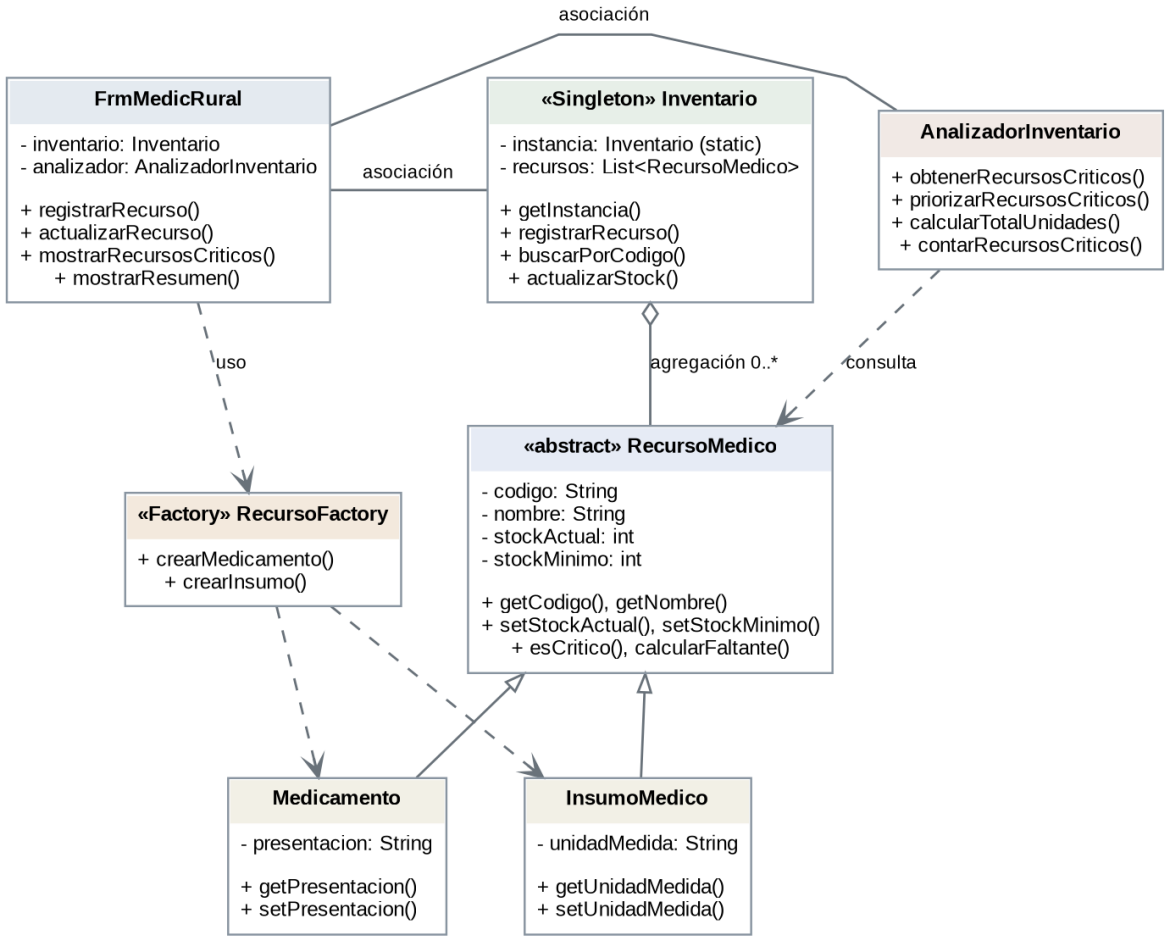
La cuarta razón fue la facilidad de mantenimiento: preferimos que Inventario se encargue de registrar y actualizar, que AnalizadorInventario se encargue de consultar, y que FrmMedicRural muestre los resultados. Esta separación permite modificar una regla sin tener que reescribir toda la ventana. Es una decisión trazable a RD04 y a los requerimientos indicados, no una arquitectura compleja creada solo para el informe.

3. Modelado del sistema con diagramas UML

3.1 Un mapa para entender las clases

Antes de programar, necesitamos saber qué partes tendrá el sistema y cómo se relacionarán. Un diagrama UML cumple esa función: es un mapa de clases. Una clase se puede imaginar como un molde que define qué información tiene un objeto y qué acciones puede realizar. La figura 1 resume las siete clases principales del proyecto, con sus atributos, operaciones y relaciones. Se ha organizado visualmente para que las conexiones se lean con mayor claridad.

Figura 1. Diagrama de clases de MedicRural-PE



3.2 Cómo se reparten las responsabilidades

RecursoMedico contiene lo que tienen en común todos los registros: código, nombre, stock actual y stock mínimo. Medicamento e InsumoMedico heredan esas características y añaden un dato propio. Inventario guarda los recursos en una lista y permite buscarlos o cambiar sus cantidades. AnalizadorInventario revisa esa lista para encontrar productos críticos, ordenarlos

y calcular totales. RecursoFactory crea cada tipo de objeto y FrmMedicRural ofrece los campos y botones que utiliza la persona usuaria.

En la figura 1 aparecen cuatro relaciones que no significan lo mismo. La herencia indica que un medicamento o insumo es un tipo de RecursoMedico. La agregación señala que Inventario reúne recursos que pueden crearse fuera de él. La asociación conecta la ventana con los objetos que necesita para funcionar. La dependencia indica un uso puntual, por ejemplo cuando la ventana llama a RecursoFactory para crear un producto. Se representan estas relaciones porque existen en el programa, no solo para decorar el diagrama.

3.3 Dos patrones de diseño usados con un propósito

Singleton resuelve una necesidad concreta: trabajar durante la ejecución con una misma instancia de Inventario. En lugar de crear una lista distinta cada vez que la ventana consulta o actualiza, el método getInstancia() devuelve el inventario compartido. Esto simplifica el prototipo, aunque no guarda datos en disco y no resuelve por sí mismo el acceso simultáneo de varios usuarios.

Factory simplifica otro paso: construir objetos. Cuando la persona elige “Medicamento” o “Insumo médico”, la ventana solicita a RecursoFactory el tipo apropiado. Así se reúne la creación de recursos en un lugar. Es una fábrica sencilla basada en métodos estáticos, no una implementación compleja de Factory Method con una jerarquía de creadores.

4. Implementación del módulo funcional operativo

4.1 Del formulario a un recurso registrado

Al abrir MedicRural-PE aparece una ventana desarrollada con Java Swing. Allí podemos seleccionar el tipo de recurso, escribir sus datos y presionar “Registrar”. El sistema convierte las cantidades escritas a números enteros, comprueba las reglas del producto, construye el objeto mediante RecursoFactory y finalmente lo agrega a Inventario. Si todo es válido, actualiza la tabla y presenta un mensaje de confirmación. Esa secuencia permite seguir la operación desde la pantalla hasta la lista donde se conserva el registro.

La información de cada producto está protegida dentro de su objeto mediante atributos privados. Para leerla se usan métodos de consulta (getters), y para modificarla se utilizan métodos que validan antes de cambiar el valor (setters). Por ejemplo, no podemos establecer un stock negativo ni un mínimo igual a cero. A esta idea la llamamos encapsulamiento: la propia clase controla cómo pueden cambiar sus datos.

4.2 Cómo decide qué producto está crítico

La regla principal es fácil de expresar: si quedan tantas unidades como el mínimo permitido, o menos, el producto se marca como crítico. Pensemos otra vez en 10 unidades disponibles y un mínimo de 20. El sistema reconoce el estado crítico y calcula un faltante de 10 unidades. Si

quedan exactamente 20, también hay una alerta de nivel mínimo, pero el faltante calculado es cero. El fragmento 1 muestra las dos operaciones que hacen esa comprobación.

Fragmento 1. Regla para detectar stock crítico y calcular faltante

```
public boolean esCritico() {
    return stockActual <= stockMinimo;
}

public int calcularFaltante() {
    if (stockActual >= stockMinimo) {
        return 0;
    }
    return stockMinimo - stockActual;
}
```

La ventaja de colocar estas reglas en RecursoMedico es que funcionan igual para los medicamentos y los insumos. No tenemos que escribir un cálculo diferente para cada producto, y podemos comprobarlo con una prueba sin abrir la ventana.

4.3 Cómo encontramos, ordenamos y resumimos productos

Una manera tradicional de buscar los productos críticos sería recorrer la lista con un ciclo y copiar a otra lista aquellos cuya cantidad esté por debajo del mínimo. En Java también podemos expresar la misma idea mediante filter: dejamos pasar únicamente los elementos que cumplen una condición. En el fragmento 2, recurso -> recurso.esCritico() significa, en palabras sencillas, “para cada producto, comprueba si es crítico”.

Fragmento 2. Selección de productos críticos mediante filter

```
public List<RecursoMedico> obtenerRecursosCriticos(
        List<RecursoMedico> recursos) {
    return recursos.stream()
            .filter(recurso -> recurso.esCritico())
            .toList();
}
```

Después de filtrar, sorted organiza los resultados del mayor al menor faltante. Por ejemplo, si faltan 25 cajas de guantes y 12 unidades de otro recurso, los guantes aparecerán primero. Esa prioridad solo representa una diferencia de cantidades; no decide qué producto es más importante para una atención médica.

Para el resumen del inventario utilizamos map y reduce. map toma el stock de cada producto y reduce va sumando esas cantidades hasta obtener un total. Son dos funciones de orden superior: aceptan otras funciones como parte de su operación. El fragmento 3 muestra el cálculo real utilizado por el sistema.

Fragmento 3. Suma de existencias con map y reduce

```
public int calcularTotalUnidades(
        List<RecursoMedico> recursos) {
    return recursos.stream()
            .map(recurso -> recurso.getStockActual())
            .reduce(0, (total, stock) -> total + stock);
}
```

Estas operaciones consultan los registros sin modificar la lista durante el recorrido. Sin embargo, los objetos siguen permitiendo actualizaciones por sus setters; por eso no afirmamos que todo el programa sea inmutable. Hemos elegido estas funciones porque facilitan la lectura de las consultas que necesita MedicRural-PE.

4.4 Qué ocurre cuando alguien presiona un botón o se equivoca

La ventana tiene seis acciones principales: registrar, actualizar, limpiar, ver todos, ver críticos y mostrar resumen. Cada botón tiene un evento que llama al método correspondiente. Un evento es simplemente algo que sucede en la interfaz, como un clic; un listener lo detecta y un handler ejecuta la respuesta. Por eso hablamos de programación orientada a eventos: el programa reacciona a las decisiones de la persona que lo utiliza.

También pensamos en errores comunes. Si alguien escribe letras donde se espera una cantidad, Integer.parseInt() no puede convertirlas y se captura NumberFormatException para mostrar un mensaje comprensible. Si se intenta registrar un código repetido, una cantidad negativa o un producto sin nombre, se usa IllegalArgumentException para explicar la regla incumplida. En vez de aceptar un dato incorrecto silenciosamente, el sistema informa qué debe corregirse.

4.5 Qué hacemos con los datos personales

El objetivo de MedicRural-PE es trabajar con productos, no con personas. Por eso la versión actual no solicita nombres de pacientes, diagnósticos ni identificadores de responsables; en la demostración deben usarse nombres de medicamentos e insumos ficticios. Reducir la cantidad de información personal solicitada es una decisión prudente y coherente con la finalidad limitada del prototipo, en relación con la Ley N.° 29733 (Congreso de la República del Perú, 2011).

Esto no significa que la aplicación tenga un sistema completo de seguridad. No incorpora cifrado, autenticación ni controles de acceso y un campo libre podría utilizarse incorrectamente para introducir información sensible. Por esa razón no debe usarse con datos clínicos reales ni presentarse como un sistema listo para operar en una institución de salud.

5. Análisis de calidad del código y buenas prácticas

5.1 Un código que podamos revisar y mejorar

La calidad del código no consiste en utilizar la mayor cantidad posible de técnicas. Para nuestro proyecto significa que otra persona pueda localizar una regla, entenderla y corregirla sin alterar funciones que no están relacionadas. Un primer ejemplo del principio de responsabilidad única (SRP) es que Inventario guarda y actualiza recursos, mientras AnalizadorInventario realiza las consultas. Si cambiamos la forma de ordenar los productos críticos, no necesitamos modificar cómo se registran.

También podemos observar el principio de sustitución de Liskov (LSP): un Medicamento o un InsumoMedico puede utilizarse donde el programa espera un RecursoMedico, porque ambos respetan las operaciones comunes, como esCritico() y calcularFaltante(). Esto permite reunir tipos distintos en una misma lista. No significa que se hayan aplicado todos los principios SOLID; aquí justificamos solamente los que tienen evidencia clara en el código.

Se reconocen además cuatro prácticas sencillas: usar nombres descriptivos como actualizarStock(); reutilizar las validaciones heredadas en vez de copiarlas; comprobar los datos antes de guardarlos; y separar las consultas de los cambios. Todas favorecen que una corrección futura sea más localizada. Hay aspectos por mejorar, como evitar que la tabla permita editar celdas sin actualizar el inventario y ampliar la documentación dentro del código.

5.2 Las pruebas: qué comprobamos y qué no

Una prueba automatizada ejecuta una regla y compara su resultado con lo que esperamos. En vez de confiar solo en que “parece funcionar”, dejamos que JUnit repita esa comprobación. La versión compartida contiene tres pruebas y tres reportes XML: cada caso registra una ejecución sin errores ni fallos. Los casos y resultados se resumen en la tabla 3. No se debe confundir que tres pruebas salgan bien con que toda la aplicación esté probada.

Tabla 3. Pruebas automatizadas existentes y resultados registrados

Caso	Qué comprueba	Evidencia
CE01 · stock crítico	Con stock 10 y mínimo 20, esCritico() devuelve verdadero.	RecursoMedicoTest: correcta
CE02 · duplicados	El código TEST001 se registra una vez; el segundo registro produce una excepción.	InventarioTest: correcta
CE03 · prioridad	Un insumo con faltante de 25 aparece antes que un medicamento con faltante de 12.	AnalizadorInventarioTest: correcta

Estas tres pruebas respaldan las reglas elegidas como criterios de éxito, incluida la combinación de clases y operaciones sobre listas. Aun así, faltan comprobaciones de la interfaz, listas vacías, cantidades límite, actualizaciones inválidas y resúmenes. Tampoco hay evidencia suficiente para afirmar que se siguió una metodología TDD desde el inicio o que existe una cobertura porcentual determinada.

5.3 Control de versiones y consecuencias para las personas

Git permite conservar un historial de cambios y saber qué se modificó. En la copia del repositorio proporcionada se identifican nueve commits locales con mensajes descriptivos en español. El desafío solicita un mínimo de diez; por tanto, con esta evidencia todavía no podemos dar ese criterio por cumplido. Antes de entregar habrá que verificar que los cambios auténticos estén publicados en el repositorio público: https://github.com/Eduard-Gozme/MedicRuralPE.

Estas precauciones no son solo técnicas. Si una regla marca incorrectamente un medicamento como disponible, una persona podría confiar en una cantidad equivocada; si se pierde el registro al cerrar, habría que volver a ingresarlo; y si alguien escribe datos personales en campos libres, podría comprometer la privacidad. Por ello, el prototipo debe acompañarse de validaciones, pruebas y una explicación clara de sus límites antes de pensar en aplicarlo fuera del aula.

6. Conclusiones, limitaciones y recomendaciones

6.1 Qué aprendimos al construir MedicRural-PE

Nuestro resultado principal es un prototipo que permite registrar y consultar medicamentos e insumos, actualizar su cantidad, reconocer productos críticos y ordenar faltantes. Elegimos una solución acotada porque necesitábamos demostrar que los conceptos del curso podían trabajar juntos en un caso comprensible. Las clases representan recursos, las funciones sobre colecciones realizan consultas y los eventos conectan esas operaciones con una ventana que puede utilizarse sin escribir código.

También comprobamos que las decisiones de diseño tienen consecuencias prácticas. La herencia evitó repetir las reglas comunes; Singleton permitió compartir el inventario durante la ejecución; y la fábrica simplificó la creación de productos. Las tres pruebas JUnit respaldan comportamientos específicos, pero no demuestran que el sistema esté listo para un establecimiento real. Precisamente por eso diferenciamos lo construido de lo que sería necesario desarrollar más adelante.

Este proyecto retoma conocimientos anteriores de programación —variables, tipos de datos, condicionales, ciclos y resolución paso a paso de problemas— y los conecta con clases, colecciones, interfaces y pruebas. Esa experiencia ofrece una base para aprendizajes posteriores relacionados con diseño de software, almacenamiento de datos y calidad. No

afirmamos una equivalencia con asignaturas específicas de la malla que no hayamos verificado (UPN, 2026b).

6.2 Qué falta y cómo lo mejoraríamos

La primera limitación es que los registros solo existen mientras la aplicación está abierta. Antes de usarla fuera de una demostración, necesitaríamos guardar los datos y comprobar que se recuperen correctamente. La segunda es que la prioridad considera el faltante numérico, no la importancia clínica, el vencimiento ni el consumo de un producto. Una mejora responsable sería revisar esas reglas con personas que conozcan el trabajo real de un establecimiento, en lugar de inventarlas desde el aula.

La tercera limitación es el alcance de las pruebas: funcionan tres escenarios, pero no toda la interfaz ni todos los casos excepcionales. Recomendamos ampliar las pruebas de cantidades límite y actualizaciones, y hacer que las celdas de la tabla sean solo de lectura para evitar confundir lo que se ve con lo que realmente está guardado. También debemos completar y publicar el historial de GitHub, además de adjuntar el formato institucional de declaración de uso de inteligencia artificial exigido por la entrega.

6.3 Nuestra responsabilidad frente a los usuarios

Un sistema de inventario puede ayudar al personal a detectar necesidades con mayor rapidez, pero no debe prometer decisiones clínicas que no puede tomar. Si en el futuro se quisiera usar en una zona rural, también habría que considerar si existen equipos adecuados, electricidad, capacitación y soporte. Algunas personas podrían beneficiarse de la organización digital, mientras que otras quedarían excluidas por falta de recursos. Como futuros profesionales, nuestra obligación es reconocer esas diferencias, proteger los datos y comprobar el funcionamiento antes de recomendar su uso.

Referencias

Congreso de la República del Perú. (2011). Ley N.° 29733, Ley de Protección de Datos Personales. https://www.leyes.congreso.gob.pe/Documentos/Leyes/29733.pdf

Equipo MedicRural-PE. (2026). MedicRuralPE [Código fuente Java]. GitHub. https://github.com/Eduard-Gozme/MedicRuralPE

Universidad Privada del Norte. (2026a). Evaluación final: Desafío del curso Lenguajes de Programación UAIN1288P, SistemaRural-PE [Documento académico del curso].

Universidad Privada del Norte. (2026b). Sílabo del curso Lenguajes de Programación UAIN1288P, período 2026-II [Documento académico del curso].